rqbit 웹 UI 통합 및 정적 파일 서빙 분석

개요

이 문서는 rqbit BitTorrent 클라이언트의 웹 UI 통합과 정적 파일 서빙 구현을 분석합니다. rqbit은 React 기반의 웹 UI를 제공하며, 빌드 시점에 정적 파일들을 바이너리에 임베딩하여 단일 실행 파일로 배포할 수 있도록 구현되어 있습니다.

웹 UI 빌드 과정 분석

1. 빌드 시스템 구조

package.json 설정

```
{
"name": "rqbit-webui",
"private": true,
"type": "module",
"scripts": {
"dev": "vite",
"build": "tsc --noEmit && vite build && node post-build",
"preview": "vite preview"
},
"dependencies": {
```

```
"@restart/ui": "^1.6.6",
"lodash.debounce": "^4.0.8",
"lodash.sortby": "^4.7",
"react": "^18.2.0",
"react-dom": "^18.2.0",
"react-icons": "^5.3.0",
"zustand": "^4.4.7"
}
}
```

특징:

- **Vite 기반**: 현대적인 프론트엔드 빌드 도구 사용
- **TypeScript**: 타입 안전성을 위한 TypeScript 사용
- **React**: 컴포넌트 기반 UI 프레임워크
- **Zustand**: 경량 상태 관리 라이브러리
- **Tailwind CSS**: 유틸리티 우선 CSS 프레임워크

Vite 설정 (vite.config.ts)

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import svgr from "vite-plugin-svgr";

export default defineConfig({
```

```
plugins: [react(), svgr()],

server: {

host: true,

port: 3031,

},

build: {

manifest: true,

},

});
```

특징:

- **매니페스트 생성**: 빌드된 파일들의 매핑 정보 생성
- **SVG 지원**: SVG 파일을 React 컴포넌트로 변환
- **개발 서버**: 포트 3031에서 개발 서버 실행

2. 빌드 후 처리 (post-build)

파일 정규화 스크립트

```
#!/usr/bin/env node


import fs from 'fs';

import path from 'path';


// Change directory to 'dist'
```

```
process.chdir('dist');


// Read and parse the manifest file
const manifestPath = path.join('.vite', 'manifest.json');
const manifest = JSON.parse(fs.readFileSync(manifestPath, 'utf-8'));


// Read the 'index.html' file
let indexHtml = fs.readFileSync('index.html', 'utf-8');


// List of files to process
const files = [
{ dst: 'assets/logo.svg', src: manifest['assets/logo.svg'].file },
{ dst: 'assets/index.css', src: manifest['index.html'].css[0] },
{ dst: 'assets/index.js', src: manifest['index.html'].file }
];


// Replace and rename files
files.forEach(({ dst, src }) => {
indexHtml = indexHtml.replace(`/${src}`, dst);
fs.renameSync(src, dst);
});


// Write the updated 'index.html'
```

```
fs.writeFileSync('index.html', indexHtml, 'utf-8');
```

목적:

- **파일명 정규화**: Vite가 생성한 해시 파일명을 고정된 이름으로 변경
- **경로 업데이트**: HTML 내의 파일 참조 경로를 새 이름으로 업데이트
- **임베딩 준비**: Rust 코드에서 `include_str!` 로 임베딩할 수 있도록 준비

3. Rust 빌드 통합 (build.rs)

자동 빌드 스크립트

```
fn main() {
#[cfg(feature = "webui")]
{
let webui_dir = Path::new("webui");
let webui_src_dir = webui_dir.join("src");

println!("cargo:rerun-if-changed={}", webui_src_dir.to_str().unwrap());

// Run "npm install && npm run build" in the webui directory
for cmd in ["npm install", "npm run build"] {
run_cmd(webui_dir, cmd).unwrap();
}
}
```

```
}
```

특징:

- **조건부 빌드**: `webui` 피처가 활성화된 경우에만 실행
- **의존성 추적**: 웹 UI 소스 변경 시 자동 재빌드
- **크로스 플랫폼**: Windows와 Unix 시스템 모두 지원
- **오류 처리**: 빌드 실패 시 상세한 오류 메시지 제공

정적 파일 서빙 구현

1. 컴파일 타임 임베딩

웹 UI 라우터 구현

```
pub fn make_webui_router() -> Router {
Router::new()
.route(
"/",
get(|| async {
(
[("Content-Type", "text/html")],
include_str!("../../webui/dist/index.html"),
)
}),
)
```

```
.route(
"/assets/index.js",
get(|| async {
(
[("Content-Type", "application/javascript")],
include_str!("../../webui/dist/assets/index.js"),
)
}),
)
.route(
"/assets/index.css",
get(|| async {
(
[("Content-Type", "text/css")],
include_str!("../../webui/dist/assets/index.css"),
)
}),
)
.route(
"/assets/logo.svg",
get(|| async {
(
[("Content-Type", "image/svg+xml")],
```

```
include_str!("../../webui/dist/assets/logo.svg"),
)
}),
)
}
```

특징:

- **컴파일 타임 임베딩**: `include_str!` 매크로로 파일 내용을 바이너리에 포함
- **적절한 MIME 타입**: 각 파일 타입에 맞는 Content-Type 헤더 설정
- **제로 카피**: 문자열 리터럴로 임베딩되어 런타임 파일 I/O 불필요
- **단일 바이너리**: 별도 파일 없이 실행 파일 하나로 배포 가능

2. 메인 라우터 통합

웹 UI 라우터 마운트

```
#[cfg(feature = "webui")]
{
use axum::response::Redirect;

let webui_router = webui::make_webui_router();
main_router = main_router.nest("/web/", webui_router);
main_router = main_router.route("/web", get(|| async { Redirect::permanent("./we
}
```

특징:

- **네임스페이스 분리**: `/web/` 경로 하위에 웹 UI 마운트
- **자동 리다이렉트**: `/web` 에서 `/web/` 로 자동 리다이렉트
- **조건부 컴파일**: `webui` 피처 플래그로 선택적 포함

3. 브라우저 자동 리다이렉트

API 루트 핸들러

```
async fn h_api_root(parts: Parts) -> impl IntoResponse {
// If browser, and webui enabled, redirect to web
#[cfg(feature = "webui")]
{
if parts
.headers
.get("Accept")
.and_then(|h| h.to_str().ok())
.is_some_and(|h| h.contains("text/html"))
{
return Redirect::temporary("./web/").into_response();
}
}

let json = serde_json::json!({
"apis": {
```

```
"GET /": "list all available APIs",
// ... API 목록
},
"server": "rqbit",
"version": env!("CARGO_PKG_VERSION"),
});

([("Content-Type", "application/json")], axum::Json(json)).into_response()
}
```

특징:

- **사용자 에이전트 감지**: Accept 헤더로 브라우저 요청 판별
- **자동 리다이렉트**: 브라우저에서 루트 접근 시 웹 UI로 리다이렉트
- **API 호환성**: API 클라이언트는 JSON 응답 수신

프론트엔드와 백엔드 간 통신

1. HTTP API 클라이언트

API URL 자동 감지

```
const apiUrl = (() => {
if (window.origin === "null") {
return "http://localhost:3030";
}
```

```
const url = new URL(window.location.href);


// assume Vite devserver
if (url.port == "3031") {
return `${url.protocol}//${url.hostname}:3030`;
}


// Remove "/web" or "/web/" from the end and also ending slash.
const path = /(.*?)\/?(\/web\/?)?$/.exec(url.pathname)![1] ?? "";
return path;
})();
```

특징:

- **개발/프로덕션 자동 감지**: 포트 번호로 환경 구분
- **상대 경로 계산**: 웹 UI 경로에서 API 경로 자동 계산
- **유연한 배포**: 다양한 경로에서 호스팅 가능

통합 요청 함수

```
const makeRequest = async (
method: string,
path: string,
data?: any,
isJson?: boolean
```

```
): Promise<any> => {
console.log(method, path);
const url = apiUrl + path;
let options: RequestInit = {
method,
headers: {
Accept: "application/json",
},
};
if (isJson) {
options.headers = {
Accept: "application/json",
"Content-Type": "application/json",
};
options.body = JSON.stringify(data);
} else {
options.body = data;
}

// 오류 처리 로직...
const result = await response.json();
return result;
```

```
};
```

특징:

- **통합 오류 처리**: 모든 API 호출에 대한 일관된 오류 처리
- **타입 안전성**: TypeScript 인터페이스로 API 응답 타입 정의
- **로깅**: 개발 시 API 호출 추적을 위한 콘솔 로그

2. 타입 정의 공유

API 타입 인터페이스

```
export interface TorrentDetails {
name: string | null;
info_hash: string;
files: Array<TorrentFile>;
}

export interface AddTorrentResponse {
id: number | null;
details: TorrentDetails;
output_folder: string;
seen_peers?: Array<string>;
}

export interface SessionStats {
```

```
counters: SessionCounters;

peers: AggregatePeerStats;

connections: ConnectionStats;

download_speed: Speed;

upload_speed: Speed;

uptime_seconds: number;

}
```

특징:

- **Rust 구조체 대응**: 백엔드 Rust 구조체와 일대일 대응
- **선택적 필드**: ? 를 사용한 선택적 필드 표현
- **중첩 구조**: 복잡한 데이터 구조의 정확한 타입 정의

3. 상태 관리

Zustand 스토어

```
export const useTorrentStore = create<TorrentStore>((set, get) => ({

torrents: [],

torrentsLoading: false,

refreshTorrents: null,

setTorrents: (torrents) => set({ torrents }),

setTorrentsLoading: (loading) => set({ torrentsLoading: loading }),

setRefreshTorrents: (refresh) => set({ refreshTorrents: refresh }),
```

```
}));
```

특징:

- **중앙 집중식 상태**: 전역 상태를 중앙에서 관리
- **반응형 업데이트**: 상태 변경 시 자동으로 UI 업데이트
- **타입 안전성**: TypeScript로 상태 구조 정의

실시간 업데이트 구현

1. 폴링 기반 업데이트

커스텀 인터벌 훅

```
const customSetInterval = (
callback: () => Promise<number>,
initialDelay: number
): (() => void) => {
let timeoutId: number | null = null;
let cancelled = false;

const scheduleNext = (delay: number) => {
if (cancelled) return;
timeoutId = window.setTimeout(async () => {
if (cancelled) return;
try {
```

```
const nextDelay = await callback();
scheduleNext(nextDelay);
} catch (error) {
console.error('Interval callback error:', error);
scheduleNext(5000); // 오류 시 5초 후 재시도
}
}, delay);
};

scheduleNext(initialDelay);

return () => {
cancelled = true;
if (timeoutId !== null) {
clearTimeout(timeoutId);
}
};
};
```

특징:

- **적응적 간격**: 콜백 함수가 다음 호출 간격을 동적으로 결정
- **오류 복구**: 오류 발생 시 자동으로 재시도
- **메모리 누수 방지**: 컴포넌트 언마운트 시 자동 정리

토렌트 목록 자동 갱신

```
useEffect(() => {
return customSetInterval(
async () =>
refreshTorrents().then(
() => {
setOtherError(null);
return 5000; // 성공 시 5초 간격
},
(e) => {
setOtherError({ text: "Error refreshing torrents", details: e });
console.error(e);
return 5000; // 오류 시에도 5초 간격
}
),
0 // 즉시 시작
);
}, []);
```

2. 실시간 로그 스트리밍

스트림 기반 로그 수신

```
const streamLogs = (
url: string,
addLine: (text: string) => void,
setError: (error: ErrorWithLabel | null) => void
): (() => void) => {
const controller = new AbortController();
const signal = controller.signal;


const runOnce = async () => {
let response = await fetch(url, { signal });


if (!response.body) {
setError({
text: "error fetching logs: ReadableStream not supported.",
});
return;
}


const reader = response.body.getReader();
let buffer = new Uint8Array();
while (true) {
const { done, value } = await reader.read();
```

```
if (done) {
setError({ text: "log stream terminated" });
throw null;
}

buffer = mergeBuffers(buffer, value);

// 줄 단위로 파싱
for (let newLineIdx: number; (newLineIdx = buffer.indexOf(10)) !== -1; ) {
let lineBytes = buffer.slice(0, newLineIdx);
let line = new TextDecoder().decode(lineBytes);
addLine(line);
buffer = buffer.slice(newLineIdx + 1);
}
}
};

// 연결 실패 시 자동 재시도 로직...
};
```

특징:

- **스트림 처리**: ReadableStream을 사용한 실시간 데이터 수신

- **버퍼 관리**: 부분적으로 수신된 데이터의 효율적 처리
- **자동 재연결**: 연결 끊김 시 자동으로 재연결 시도
- **취소 가능**: AbortController를 통한 스트림 취소 지원

3. WebSocket 대안

rqbit은 WebSocket 대신 Server-Sent Events(SSE) 스타일의 HTTP 스트리밍을 사용합니다:

백엔드 로그 스트리밍 엔드포인트

```
pub async fn h_stream_logs(
State(state): State<ApiState>,
) -> Result<impl IntoResponse> {
let stream = state
.api
.api_log_lines_stream()
.context("log streaming not available")?;
Ok(axum::body::Body::from_stream(stream))
}
```

특징:

- **단방향 스트림**: 서버에서 클라이언트로의 단방향 데이터 전송
- **HTTP 호환성**: 표준 HTTP 연결 사용으로 프록시 친화적
- **백프레셔 지원**: 클라이언트 처리 속도에 맞춘 데이터 전송

개발 환경 지원

1. 개발 서버 설정

CORS 설정

```
const ALLOWED_ORIGINS: [&[u8]; 4] = [
// Webui-dev
b"http://localhost:3031",
b"http://127.0.0.1:3031",
// Tauri dev
b"http://localhost:1420",
// Tauri prod
b"tauri://localhost",
];
```

특징:

- **개발 환경 지원**: Vite 개발 서버와 Tauri 개발 환경 지원
- **유연한 CORS**: 환경 변수를 통한 추가 오리진 허용
- **보안**: 프로덕션에서는 필요한 오리진만 허용

2. 핫 리로드 지원

개발 시에는 Vite 개발 서버가 핫 리로드를 제공하고, 프로덕션에서는 임베딩된 파일을 사용합니다.

성능 최적화

1. 번들 크기 최적화

트리 셰이킹

- **ES 모듈**: Vite의 트리 셰이킹으로 사용되지 않는 코드 제거
- **선택적 임포트**: 필요한 컴포넌트만 임포트하여 번들 크기 최소화

코드 분할

```
// 지연 로딩 예시 (실제 구현에서는 사용되지 않음)
const LazyComponent = React.lazy(() => import('./LazyComponent'));
```

2. 캐싱 전략

정적 파일 캐싱

- **컴파일 타임 임베딩**: 파일 내용이 바이너리에 포함되어 디스크 I/O 없음
- **메모리 효율성**: 문자열 리터럴로 저장되어 메모리 사용량 최소화

3. 네트워크 최적화

API 호출 최적화

- **배치 요청**: 여러 토렌트 정보를 한 번에 가져오기
- **조건부 요청**: 변경된 데이터만 업데이트
- **오류 시 백오프**: 네트워크 오류 시 점진적으로 재시도 간격 증가

보안 고려사항

1. XSS 방지

React의 기본 보호

- **자동 이스케이핑**: React가 자동으로 사용자 입력 이스케이프
- **dangerouslySetInnerHTML 금지**: 직접 HTML 삽입 방지

2. CSRF 방지

동일 출처 정책

- **CORS 제한**: 허용된 오리진에서만 API 접근 가능
- **기본 인증**: HTTP Basic Auth를 통한 접근 제어

3. 콘텐츠 보안

적절한 MIME 타입

- **Content-Type 헤더**: 각 파일 타입에 맞는 MIME 타입 설정
- **브라우저 보안**: 브라우저의 내장 보안 기능 활용

결론

rqbit의 웹 UI 통합은 다음과 같은 특징을 보여줍니다:

1. **단일 바이너리 배포**: 컴파일 타임 임베딩으로 별도 파일 없이 배포
2. **현대적 프론트엔드**: React, TypeScript, Vite를 사용한 현대적 개발 환경
3. **실시간 업데이트**: 폴링과 스트리밍을 통한 실시간 데이터 업데이트
4. **개발자 친화적**: 핫 리로드와 타입 안전성을 제공하는 개발 환경
5. **성능 최적화**: 번들 크기 최소화와 효율적인 네트워크 사용
6. **보안**: XSS, CSRF 방지와 적절한 접근 제어

이러한 설계는 BitTorrent 클라이언트의 복잡한 상태를 직관적인 웹 인터페이스로 제공하면서도, 배포와 유지보수의 편의성을 극대화합니다.